Git & GitHub – 50 Commands Every Beginner Should Know

1. `git init`

   Creates a new Git repository in the current folder.
   Starts tracking files using Git.

2. `git clone <url>`

   Copies a remote repository to your local system.
   Used to start working on an existing project.

3. `git status`

   Shows current branch and file states.
   Tells what is modified, staged, or untracked.

4. `git add <file>`

   Moves a file to the staging area.
   Prepares changes for commit.

5. `git add .`

   Stages all changed files at once.
   Used before committing multiple changes.

6. `git commit -m 'message'`

   Creates a snapshot of staged changes.
   Message explains what you changed.

7. `git log`

   Shows full commit history.
   Helps you see past changes.

8. `git log -oneline`

   Shows short commit history.
   Easy to read and quick.

9. `git diff`

   Shows what changed but not staged. Used to review before adding.

10. `git diff -staged`

    Shows staged changes only.
    Used to review before committing.

Git & GitHub –

50 Commands Every Beginner Should Know

11. `git branch`

Shows all local branches. | Highlights your current branch.

12. `git branch -a`

Shows local and remote branches.
Helps you see what exists on GitHub.

13. `git branch <name>`

Creates a new branch.
Does not switch to it.

14. `git checkout <branch>`

Switches to another branch.
Moves your working area.

15. `git switch <branch>`

Modern way to change branch. | Safer than checkout.

16. `git checkout -b <branch>`

Creates and switches to a new branch.
One command for two actions.

17. `git merge <branch>`

Merges another branch
Into current.

18 `git branch -d <branch>`

Deletes a local branch.
Used after merge.

19. `git fetch`

Downloads changes from remote.
Does not merge them.

20. `git pull`

Fetch + merge in one step.
Updates your local branch.

20. `git pull`

Fetch + merge in one step.
Updates your local branch.

Git & GitHub –

50 Commands Every Beginner Should Know

21. `git remote -v`

Shows linked remote repositories.
Displays fetch and push URLs.

22. `git remote add origin <url>`

Connects local repo to GitHub.
Names the remote as “origin”.

23. `git push origin <branch>`

Uploads commits to GitHub.
Shares your work.

24. `git push -u origin <branch>`

Pushes and sets upstream.
Future push works without branch name.

25. `git pull origin <branch>`

Downloads and merges from GitHub.
Keeps you updated.

26. `git stash`

Temporarily saves
Cncommitted work.

27. `git stash pop`

Restores last stashed work.
Removes it from stash list.

27. `git stash list`

Shows all saved stashes.
Helps you choose.

28. `git stash list`

Shows all saved stashes.
Used after merge.

29. `git reset HEAD <file>`

Unstages a file.
Keeps changes.

30. `git restore <file>`

Discards local changes.
Restores last committed version.

Git & GitHub – 50 Commands Every Beginner Should Know

31. `git reset --hard`

Resets files and commits to last saved state.
Deletes all uncommitted changes.

32. `git reset --soft HEAD~1`

Removes last commit but keeps changes staged.
Used to edit commit message or content.

33. `git revert <commit>`

Creates a new commit that undoes another commit.
Safe for shared branches.

34. `git cherry-pick <commit>`

Applies one commit from another branch.
Used to copy fixes.

35. `git rebase <branch>`

Moves your commits on top of another branch.
Creates clean history.

36. `git rebase --abort`

Stops rebase and returns to old state.
Used when conflicts are confusing.

37. `git merge --abort`

Cancels an ongoing merge.
Restores previous state.

38. `git merge --abort`

Cancels an ongoing merge.
Restores previous state.

38. `git diff main..feature`

Shows difference between two branches. Used before PR.

39. `git show <commit>`

Shows details of one commit.
Includes code and message.

40. `git tag v1.0`

Creates a version label.
Used for releases.

Git & GitHub –

50 Commands Every Beginner Should Know

41. `git tag -a v1.0 -m "version 1.0"`

Creates an annotated version tag.
Used to mark releases clearly.

42. `git push origin --tags`

Uploads all tags to GitHub.
Shares release versions.

43. `git show--branch`

Shows branch history summary.
Helps understand merges.

44. `git blame <file>`

Shows who changed each line.
Used for debugging.

45. `git clean -f`

Deletes untracked files.
Cleans workspace.

46. `git clean -fd`

Deletes untracked files
Cleans workspace.

47. `git clean –fd`

Deletes untracked files and folders. Use carefully.

49. `git reflog`

Shows all HEAD movements.
Helps recover lost commits.

50. `git archive`

Creates zip/tar of repo.
Used to share source without Git history.

47. `git config --list`

Shows all Git settings.
Used to debug config.

48. `git config --global --edit`

Opens global config file.
Edit username, email, etc.

49. `git archive`

Creates zip/tar of repo.
Used to share source without Git history.

50. `git archive`

Creates zip/tar of repo.
Used to share source without Git history.